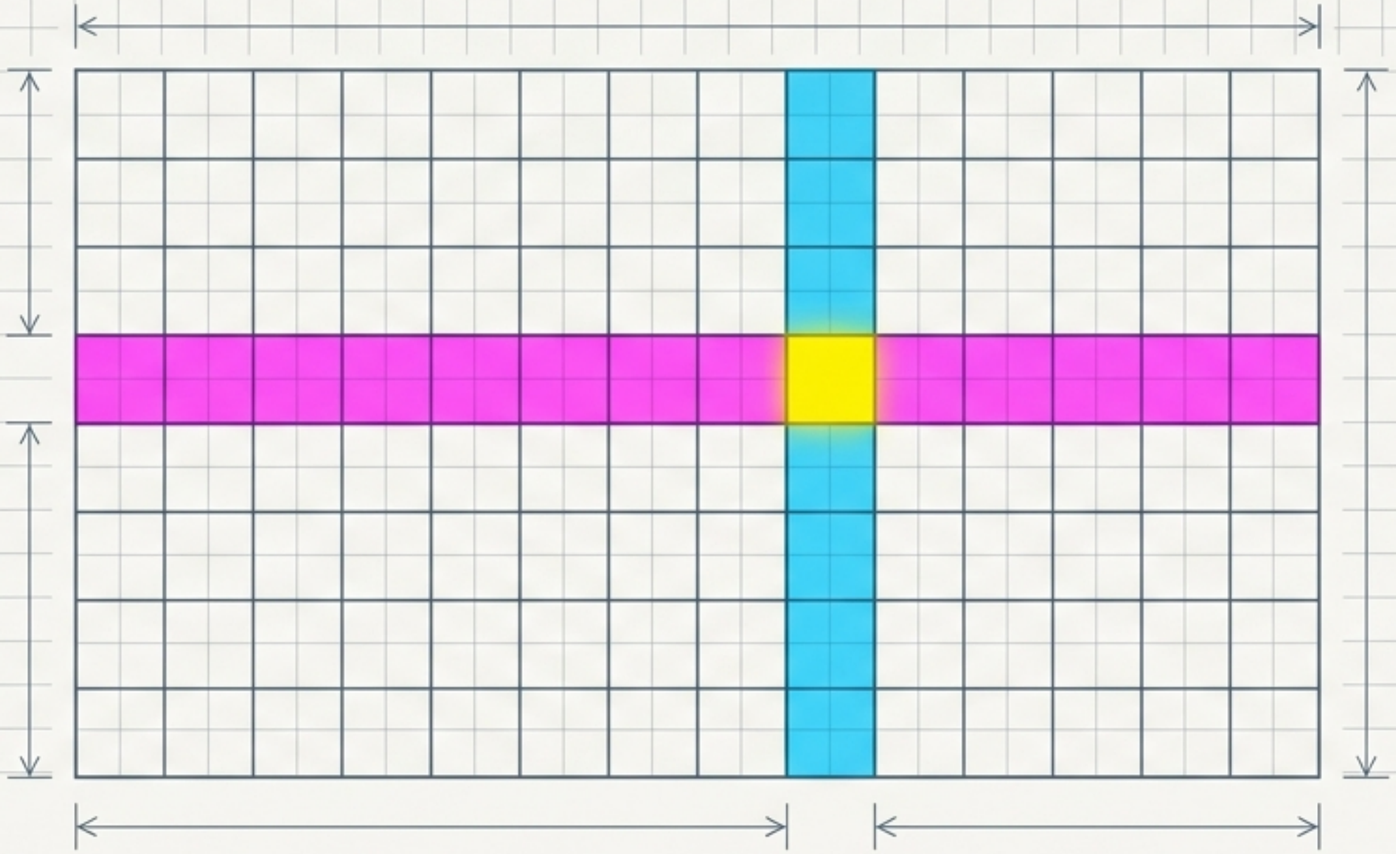


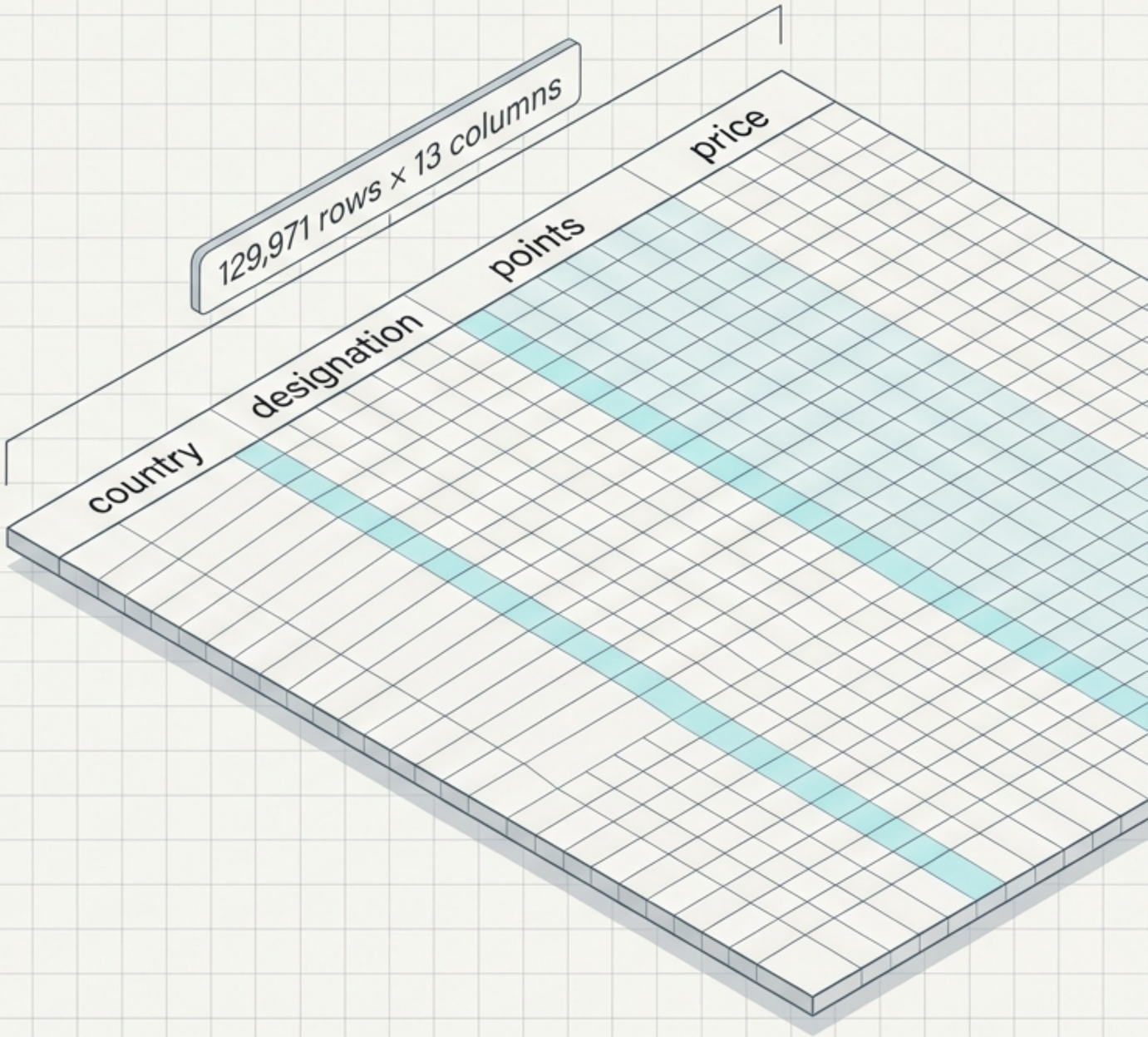
Navigating the Pandas DataFrame

A masterclass in indexing, selecting, and assigning data.

Selecting specific data points is a spatial navigation problem.

Before performing any data operation, you must isolate the exact coordinates of the relevant data.

Pandas provides a map—the DataFrame—and a progression of tools to extract value from it, ranging from basic street names to precise GPS targeting and logical filtering.



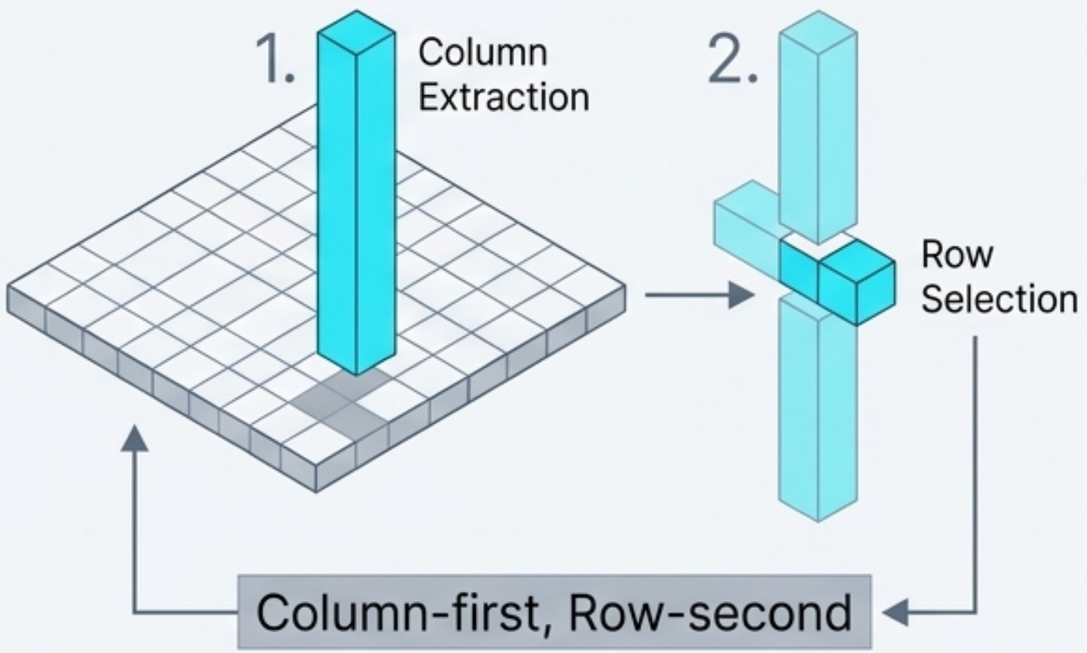
Native Python accessors act as street names for your columns.

Pandas carries over native Python object and dictionary indexing. Attribute access (.) is fast and familiar, while dictionary indexing ([ ]) safely handles column names with spaces or reserved characters.

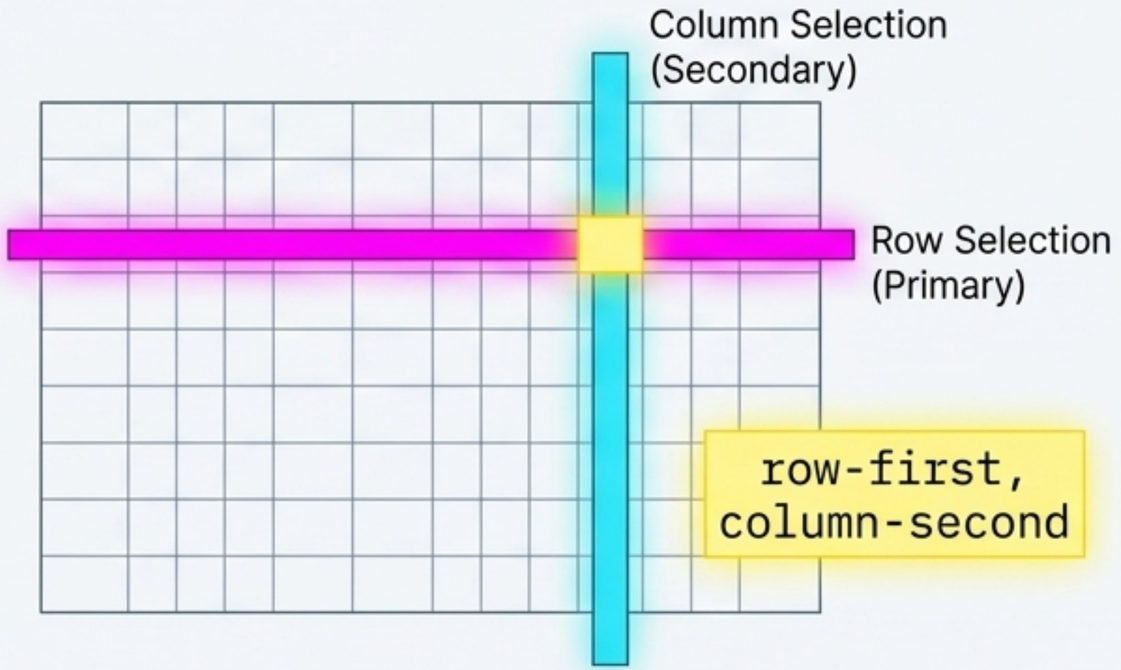
Pandas introduces specialized targeting for the two-dimensional grid.

Native accessors are excellent for broad column selection, but advanced operations require the **loc** and **iloc** accessors. Unlike native Python, Pandas accessors are strictly row-first, column-second, prioritizing the extraction of entire records.

Native Python Strategy

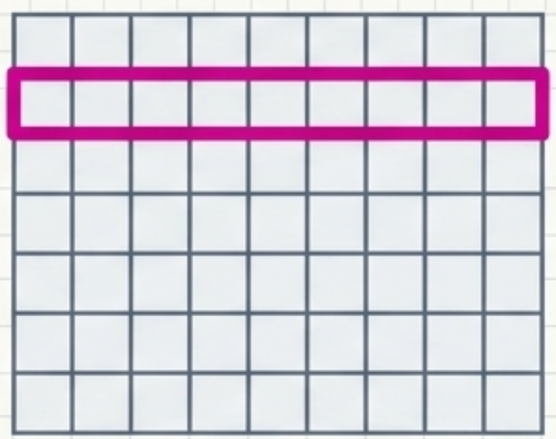


Pandas Native Strategy

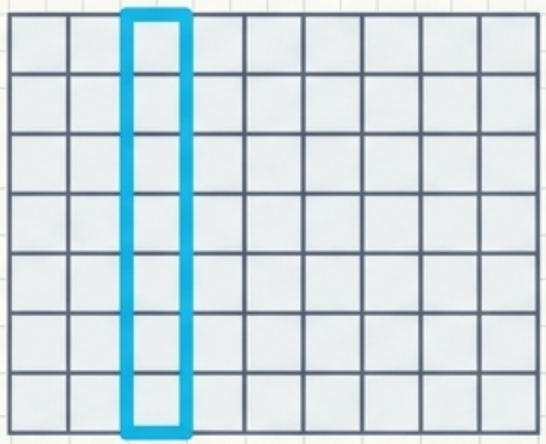


`iloc` finds exact GPS coordinates via numerical position.

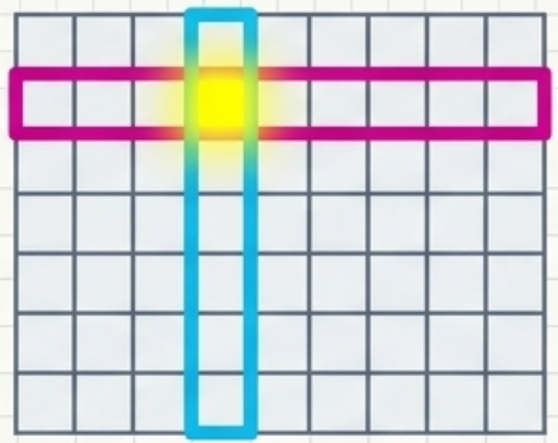
Index-based selection (iloc) treats the DataFrame like a massive **matrix**. It completely **ignores** labels, allowing you to select data purely based on its **numerical position**, including ranges and negative numbers counting from the end.



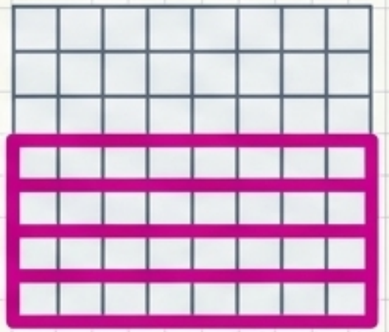
```
reviews.iloc[0]
```



```
reviews.iloc[:, 1]
```



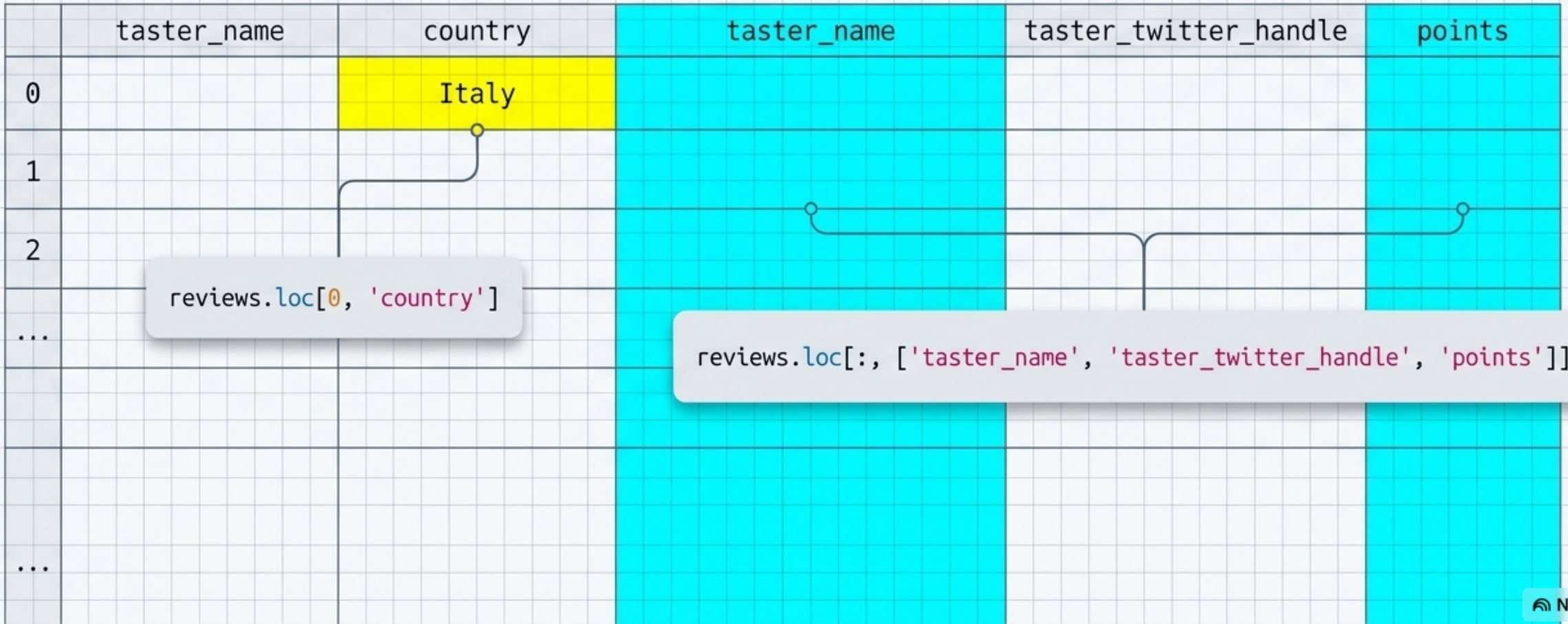
```
reviews.iloc[0, 1]
```



```
reviews.iloc[-5:]
```

loc locates data by its named postal address.

Label-based selection (loc) operates on the data's index values, not its position. Since real-world datasets have meaningful indices, loc is usually the most powerful way to retrieve specific, named properties.



The Targeting Matrix reveals the core operational differences.

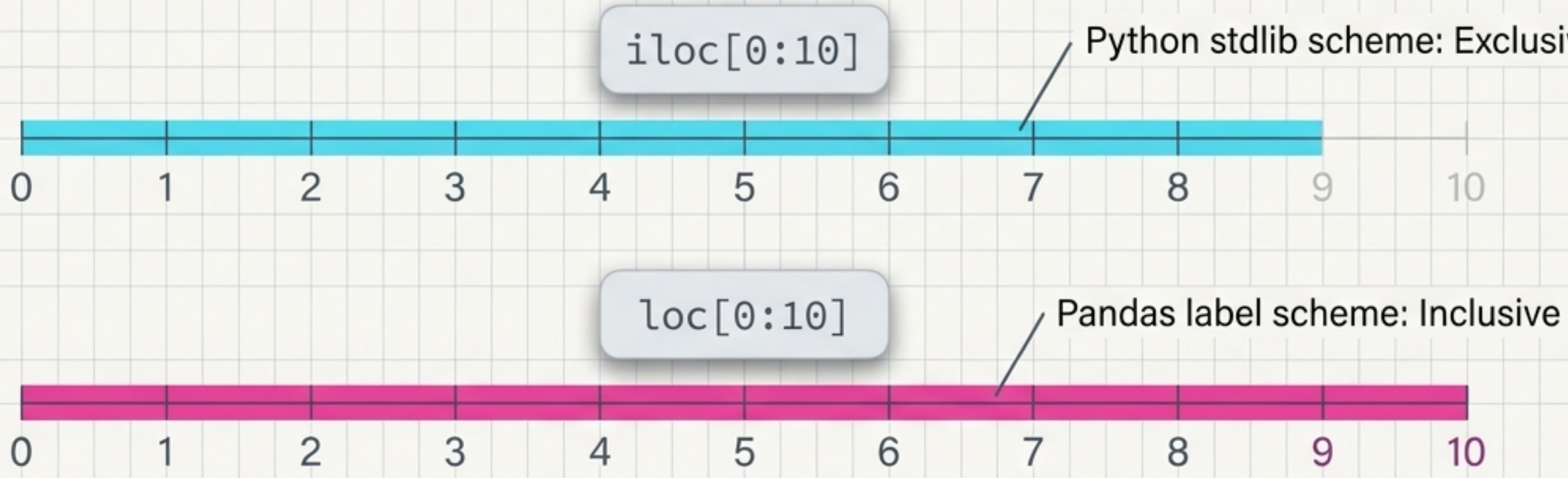
Choosing between loc and iloc dictates how you speak to the DataFrame. Memorize these distinct behavioral profiles.

The Targeting Matrix

Paradigm Name	Mental Model	Target Type	Slicing Behavior
iloc (Index-based)	Big Matrix (list of lists)	Numerical Position	Exclusive end (0 to 9)
loc (Label-based)	Fancy Dictionary	Data Index Value	Inclusive end (0 to 10)

Slicing behavior changes depending on your targeting paradigm.

The most common pitfall in Pandas indexing is range mismatch. `iloc` follows standard Python rules, excluding the final element. `loc` is inclusive, meaning `reviews.loc[0:1000]` will return 1,001 entries, not 1,000.



If indexing strings (Apples to Potatoes), an inclusive end prevents you from having to guess the next alphabetical character.

The index is not immutable and can be rebuilt on demand.

Label-based selection derives its power from the labels. If a different column provides better identifiers for your records, you can physically lock it into place as the new anchor of the dataset.

Index Mutation Flow

```
reviews.set_index('title')
```

State A

	...	title	score	date	author
0	...	Data Analysis 101	18	2023-11-26	jouy
1	...	Machine Learning Intro	13	2021-10-07	iṁlw
2	...	Advanced Pandas	15	2012-13-05	jayan
3	...	Advanced Learning	8	2011-03-07	carktorman
4	...	Tasking hiptorl	5	2011-03-03	stasson
...	...	...	...	...	...

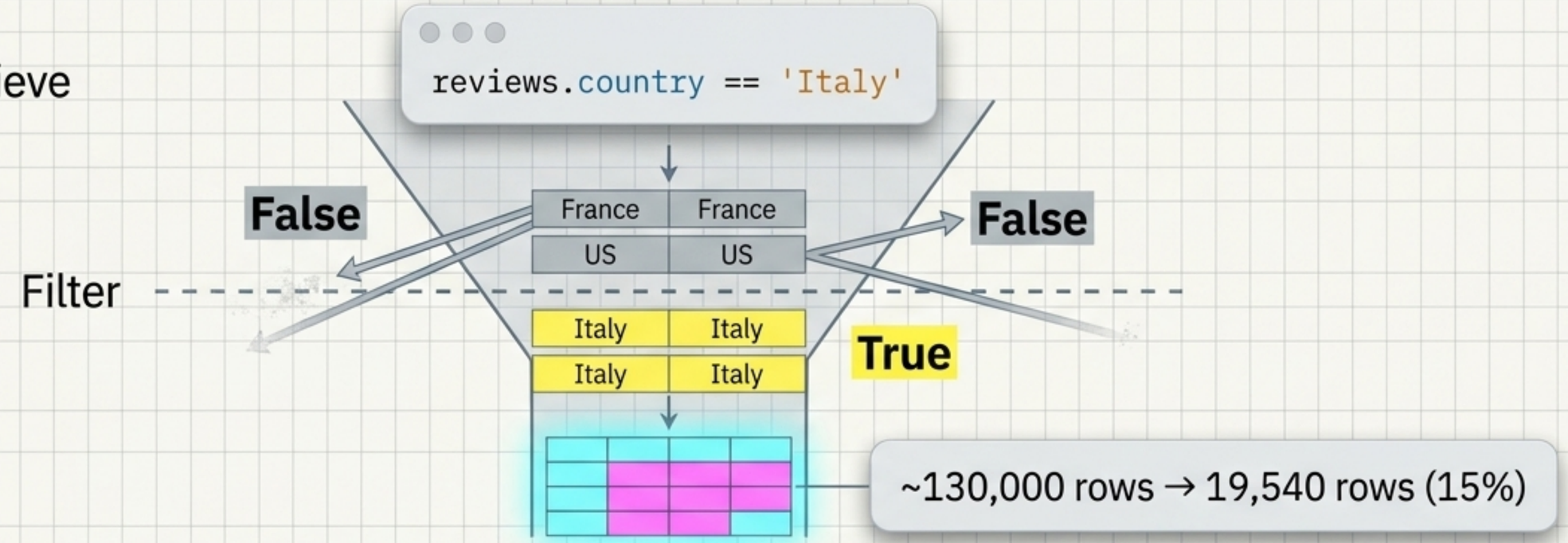
State B

	title	score	date	author
0	Data Analysis 101	18	2023-10-26	jouy
1	Machine Learning Intro	13	2021-10-07	lo̅w
2	Advanced Pandas	15	2012-16-05	jayan
3	Advanced Learning	8	2011-04-07	cantoman
4	Tasking hiptorl	5	2011-03-03	stasson
...	...	...	...	...

Conditional selection filters the map to relevant territories.

Asking questions of your data generates a Series of True/False booleans. Passing this boolean map into `loc` instantly strips away the noise, revealing only the records that meet your criteria.

Boolean Sieve

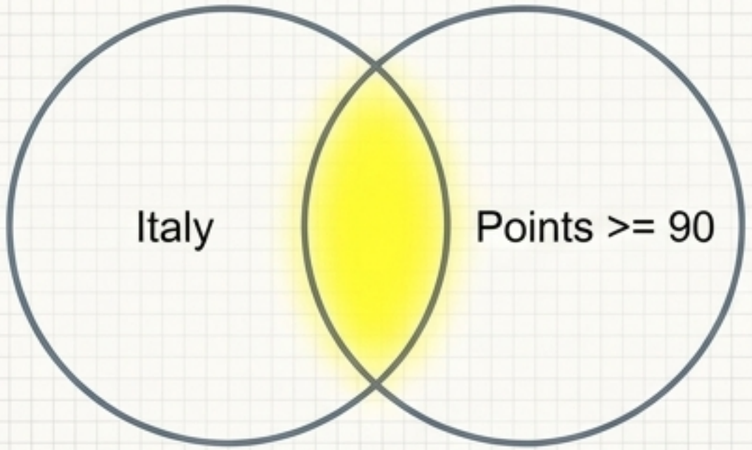



```
reviews.loc[reviews.country == 'Italy']
```

Combine analytical questions using logic gates

Real-world queries rarely rely on a single condition. Use the ampersand to demand both criteria be met (intersection), or the pipe to accept either criterion (union).

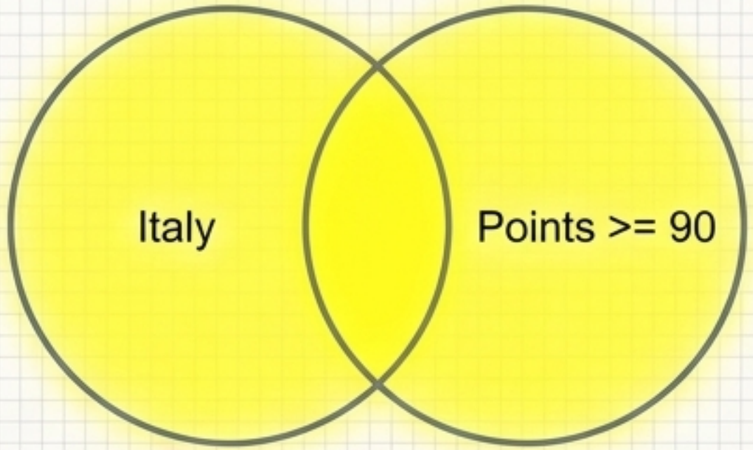
The Ampersand & / AND




```
reviews.loc[(reviews.country == 'Italy') &
            (reviews.points >= 90)]
```

Fewer results: 6,648 rows

The Pipe | / OR



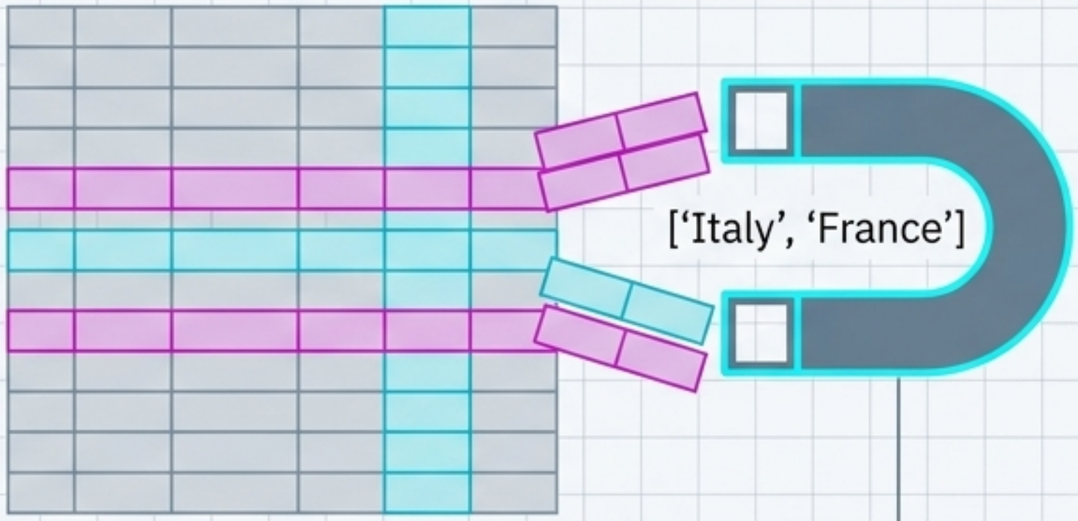

```
reviews.loc[(reviews.country == 'Italy') |
            (reviews.points >= 90)]
```

More results: 61,937 rows

Built-in selectors isolate specific lists or empty data.

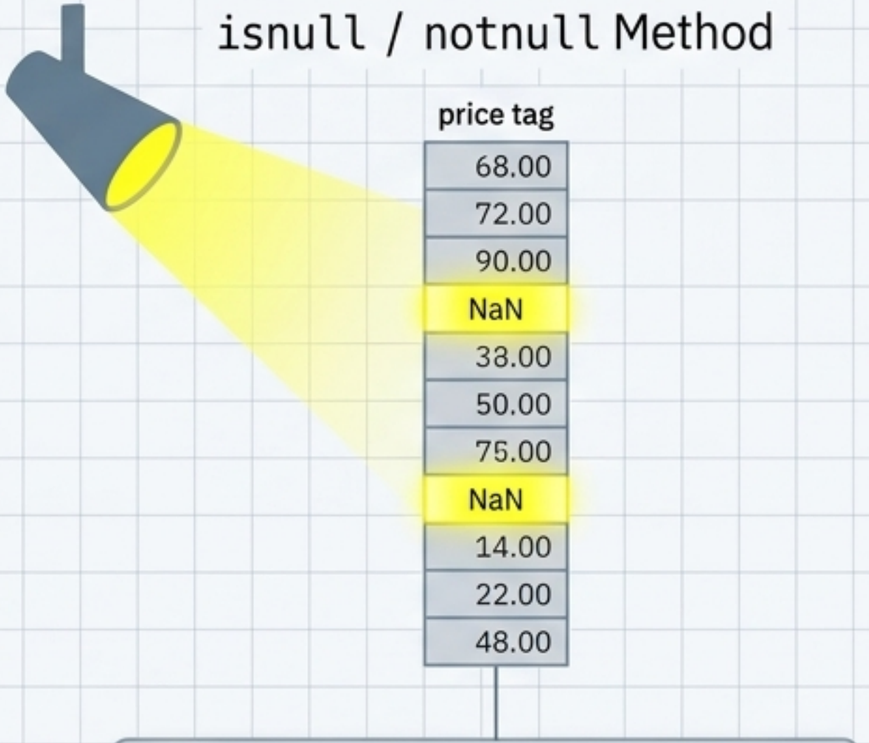
Pandas includes specialized methods to streamline complex queries. `isin` allows you to select data matching an exact list of values, while `isnull` and `notnull` cleanly separate populated data from missing values.

`isin` Method




```
reviews.loc[reviews.country.isin(['Italy', 'France')]]
```

`isnull` / `notnull` Method




```
reviews.loc[reviews.price.notnull()]
```

Assigning data instantly reshapes the underlying topography.

Selection is the prerequisite for manipulation. Once you define a column, you can flood it with a constant value or map an iterable array of values perfectly into its structure.

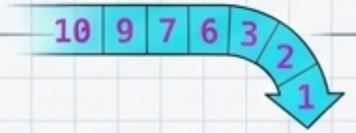
Constant Value Assignment

country	points	price	critic		...
country	13	$70	...		
USA	10	$100	...	'everyone'	
country	13	$78	...	'everyone'	
USA	12	$120	...	'everyone'	
Spain	13	$120	...	'everyone'	
Italy	14	$110	...	'everyone'	
Spain	11	$110	...	'everyone'	
Brazil	12	$30	...	'everyone'	
Spann	13	$80	...	'everyone'	
Teerzn	19	$100	...	'everyone'	
country	17	$70	...	'everyone'	
USA	22	$150	...	'everyone'	
Germany	27	$160	...	'everyone'	

```
reviews['critic'] = 'everyone'
```

```
reviews['index_backwards'] =
        range(len(reviews), 0, -1)
```

Iterable Assignment



country	points	price	...	...	index_backwards	...
country	10	$75	...		10	
USA	13	$120	...		9	
Germany	15	$120	...		8	
USA	19	$100	...		7	
Spain	13	$130	...		6	
Italy	22	$110	...		5	
Frann	18	$90	...		4	
country	17	$85	...		3	
USA	22	$90	...		2	
Germany	27	$160	...		1	

The Data Navigator's Cheat Sheet.

Master these four paths, and you possess the exact syntax required to extract any insight from a DataFrame.